

Guía de Aprendizaje: Memoria Compartida (Shared Memory)

Malak Sanchez

Monitorías de Sistemas Operativos

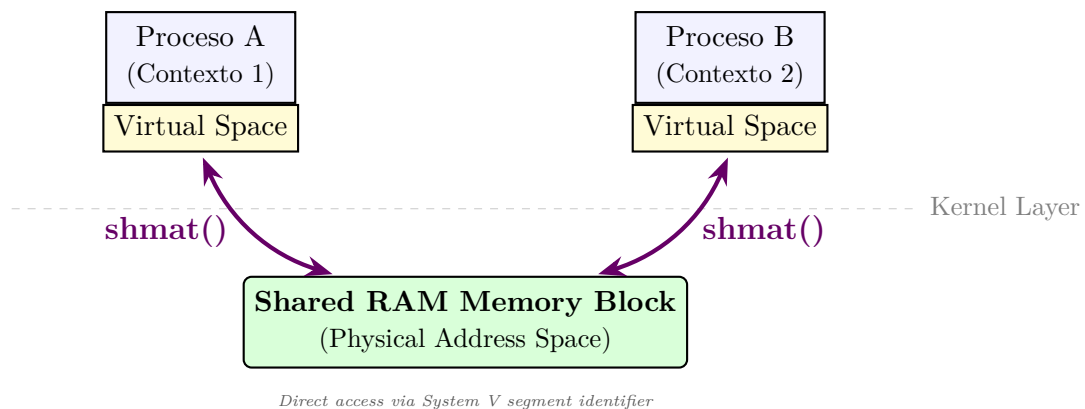
24 de abril de 2026

1. Motivación

Cuando necesitamos transferir grandes volúmenes de datos entre procesos de manera ultra-rápida, los pipes son lentos porque implican copiar datos entre el espacio de usuario y el kernel. La **Memoria Compartida** permite que dos procesos mapeen la misma región de memoria física a su espacio de direcciones virtual.

2. Idea Fundamental

Normalmente, el kernel garantiza que un proceso no pueda acceder a la memoria de otro. La memoria compartida es una excepción controlada:



1. Se crea un segmento de memoria en el kernel.
2. Cada proceso lo "mapea" (adjunta) a su propio espacio virtual.
3. Los procesos leen y escriben como si fuera memoria local normal.

3. Sintaxis (Estilo System V)

```
#include <sys/shm.h>
#include <sys/ipc.h>

// 1. Get identifier (Create segment)
int shmid = shmget(key, size, IPC_CREAT | 0666);

// 2. Attach to process address space
void *ptr = (void *)shmat(shmid, NULL, 0);

// 3. Detach when finished
shmdt(ptr);

// 4. Control/Removal
shmctl(shmid, IPC_RMID, NULL);
```

Descripción de las operaciones

- **shmget():** Crea u obtiene un segmento de memoria compartida identificado por una clave (*key*). Devuelve un identificador del segmento (*shmid*) que será utilizado en las operaciones posteriores.
- **shmat():** Adjunta el segmento de memoria compartida al espacio de direcciones del proceso y devuelve un puntero que permite acceder a dicha memoria como si fuera memoria normal.
- **shmdt():** Desvincula el segmento de memoria compartida del proceso una vez que este ha terminado de utilizarlo.
- **shmctl():** Permite realizar operaciones de control sobre el segmento; comúnmente se usa con `IPC_RMID` para marcar el segmento para su eliminación del sistema.

4. Ejemplo: Contador Compartido

Este ejemplo muestra la creación de un segmento para un entero compartido que es modificado por un proceso padre y su hijo simultáneamente.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <sys/shm.h>
6 #include <sys/stat.h>
7
8 int main() {
9     int shmid, *counter;
10    int max_iter = 1000000;
11
12    // Create shared memory segment (System V)
13    shmid = shmget(IPC_PRIVATE, sizeof(int), 0666 | IPC_CREAT);
14    counter = (int *)shmat(shmid, NULL, 0);
15    *counter = 0;
16
17    if(fork()){ // Parent process
18        for(int i=0; i < max_iter; i++){
19            *counter = *counter + 1;
20        }
21
22        wait(NULL);
23        printf("Final value of counter: %d\n", *counter);
24
25        shmdt(counter); // Detach from process address space
26        shmctl(shmid, IPC_RMID, NULL); // Remove shared memory
27                                     segment
28    }else{// Child process
29        for (int i = 0; i < max_iter; i++) *counter = *counter +
30            1;
31
32        shmdt(counter); // Detach from process address space
33    }
34
35    return EXIT_SUCCESS;
36 }
```

5. Precaución: El Problema de la Carrera

La memoria compartida **no tiene sincronización automática**. Si ambos procesos escriben al mismo tiempo (como en el ejemplo anterior), el resultado final podría no ser el esperado debido a condiciones de carrera. Para esto necesitaremos algún mecanismo de sincronización.

6. Ejercicios

1. Ejecute el ejemplo del contador varias veces. ¿Por qué el resultado varía o no llega al valor esperado?
2. Investigue la diferencia entre `IPC_PRIVATE` y usar una clave generada por `ftok()`.
3. Investigue el comando `ipcs -m` en Linux.